CAPSTONE DEFENSE / SOURCE-CHECKED EDITION

ElderCareAI System & Panel Defense Handbook

Features, architecture, Google Health webhook, role-based access, database rules, AI, safety limits, deployment, tests, and likely panel questions.

The 30-second answer
ElderCareAI helps a caregiver track one linked older adult's available Google Health readings alongside medications, appointments, notes, alerts, and care records. An optional Gemini-based assistant explains saved information. A separate admin website manages accounts and system records. Supabase provides authentication, PostgreSQL, RLS, Edge Functions, Storage, and Realtime.

Use this part	For
Pages 2-7	Learn the system, data flow, security, features, and limitations.
Question bank	Practice concise spoken answers. Each answer names code evidence with [S#] references.
Final checklist	Choose a defensible live demonstration and show what was actually verified.

Evidence standard
This handbook describes the checked-out repository at commit **a332541**. Code implementation is verified by inspection. Live Google Health subscriber registration, deployed function versions, real-device notification behavior, and production availability were **not** independently verified for this handbook. Say 'implemented in code' unless you have deployment evidence.

Prepared 27 October 2026, Asia/Manila. This is a presentation and study aid, not clinical guidance.

1 System at a glance

Problem. Caregivers often need one place to review available wearable information and everyday care tasks. The app brings those records together and makes the age of each measurement visible. It cannot guarantee a watch measurement or replace medical assessment.

Layer	Implemented technology	Responsibility
Mobile	Expo SDK 54, React Native 0.81.5, TypeScript, Expo Router	Caregiver screens, local reminders, phone location, PDF export.
Backend	Supabase Auth, PostgreSQL, RLS, Realtime, Storage, Edge Functions	Identity, records, authorization, secure server calls, event updates.
External	Google Health API v4; Gemini API	Source of wearable measurements; Elle text responses.
Web	Static public site and /edcadmin/ portal	Project information and privileged administration.
Build/test	EAS, Jest, React Native Testing Library, TypeScript	APK/build workflow and automated logic/component checks.

The five-step data path

1	2	3	4	5
Wearable records	Phone/Google Health uploads	Webhook or app sync requests	Supabase saves changed values	Open app receives Realtime event

Three clocks to distinguish
Measured time: when the health sample was recorded. **Watch last-sync time:** when Google Health reports the paired tracker last synced. **Server check / synced_at:** when ElderCareAI last queried Google Health. A new check can return an old sample.

Main sources: [S1], [S2], [S3], [S4], [S5].

2 Caregiver journey and feature inventory

Stage / screen	What the caregiver can do	Important boundary
Google sign-in	Sign in via Supabase Google OAuth; restore the session on return.	Requires configured OAuth redirects and network.
Onboarding	Complete intro, caregiver and older-adult profiles, optional Health connection, location choice.	Progress persists and resumes at the next unfinished step.
Home	See patient summary, six available vital metrics, care preview, doctor shortcut, connection and last-sync status.	Missing values remain empty; timestamps show freshness.
Health	Open vital details and sleep history; review insights, trend projections, anomalies.	Statistics use available history and are non-diagnostic.
Care	Manage medications, taken-dose logs, appointments, caregiver notes.	Reminders are scheduled on the device.

Stage / screen	What the caregiver can do	Important boundary
Alerts	Review health alerts, notifications and SOS history; initiate SOS with confirmation.	SOS logs and opens a phone call; it is not dispatch.
Elle	Ask a Gemini-backed assistant about authorized saved context.	Can be unavailable and must not prescribe/diagnose.
Settings/records	Edit profiles/doctor, manage connection, theme, permissions, checkups, and share PDF reports.	Caregiver-entered records are not certified clinical records.

Sources: [S2], [S6], [S7], [S8], [S9], [S10].

3 Authentication, Google Health connection and webhook

Two different OAuth flows

- **App sign-in:** Supabase Google OAuth uses PKCE and returns to **eldercareai://auth/callback**. The app exchanges the code for a persisted caregiver session and resumes onboarding.
- **Google Health authorization:** the signed-in caregiver calls an Edge Function, which creates a short-lived state and PKCE verifier. Google returns to the public callback, which consumes the state, exchanges the code server-side, obtains the Google Health user ID, stores OAuth tokens in a server-protected table, and starts an initial sync.

Webhook processing - the answer to rehearse

Step	Code behavior
Registration	A Google Health AUTOMATIC subscriber must point to the public HTTPS Edge Function endpoint and use the configured shared authorization secret. Registration and user consent are deployment tasks.
Gate	The webhook accepts POST only. It compares the Authorization header with a server secret. A correctly authorized verification event returns 201; an unauthorized call returns 401.
Authenticity	For actual notifications it checks Google's signature over the raw body using Google's rotating public keyset and ECDSA P-256/SHA-256.
Filter/map	It accepts UPSERT/DELETE for the seven watched data types, deduplicates healthUserId values, and finds the matching protected OAuth token row.
Sync	It refreshes the token if needed, requests current Google Health data, builds a consolidated snapshot, and writes a vital row only when a value, timestamp, or sleep history changes.
UI	PostgreSQL Realtime sends a changed vital row to the open app. Closed apps see saved data on the next launch/foreground load.

Delivery and failure behavior
The handler returns 204 on success, 403 for missing/invalid event signatures, and 503 when signature verification or sync cannot complete. No request can make the watch upload a sample. The app also checks on launch/foreground, offers manual Sync now, and has a foreground stale-reading retry policy. These are recovery paths, not a guaranteed live stream.

Sources: [S3], [S4], [S5], [S11], [S12], [S13].

4 Roles, RLS and admin security

Identity	Allowed path	Enforced by
Active caregiver	Own linked patient and care data in the mobile app.	Supabase session + ownership RLS + active-caregiver restrictive policies.
Suspended caregiver	No protected caregiver table access, even with an unexpired JWT.	Live user_access status in is_active_caregiver() RLS checks; app also signs out at its next check.
Active admin	Admin portal operations across users and reports.	Separate Auth user + active admin user_access row + Edge Function recheck on every request.
Suspended admin	No admin API access.	Admin Edge Function checks the current database role/status.
Anonymous/ordinary browser	Public site and public assets only.	No service-role key in browser; sensitive tables require auth and policies.

Defense-level explanation

- **RBAC** decides the broad role: caregiver or admin. The database stores it in **user_access** with active/suspended status; new Auth users default to caregiver.
- **RLS** decides which rows an authenticated caregiver can reach. Ownership policies match the caregiver ID or linked older-adult ID. A later restrictive policy ANDs active-caregiver status with these policies.
- The admin portal does not use a hidden URL as protection. It sends a bearer token to **admin-portal**; that function validates the user, reloads role/status, then performs privileged work with its server-only service key.
- The portal keeps its token only in page memory and clears it after 15 minutes idle. Admin changes/exports are written to **admin_audit_log**. The database prevents removal of the final active admin.

Be candid about privacy
The **profile-photos** Storage bucket is public. Previously published photo URLs may remain reachable after suspension. Health tokens and admin audit records are protected from ordinary authenticated clients. Do not claim every stored asset is private.

Sources: [S14], [S15], [S16], [S17], [S18].

5 Data, analysis, reminders and boundaries

Core records

Group	Tables / fields	What to remember
Identity	auth.users, profiles, caregivers, user_access, onboarding_progress	Auth identity is separate from app profile and access status.
Patient	elderly_profiles, doctor_contacts	Current sync implementation selects the caregiver's first older-adult profile.
Health	google_health_tokens, google_health_oauth_states, vital_sign_logs, wearable_sync_locations	Tokens are server-only; vitals carry measurement_times and synced_at separately.
Care	medication_schedules/logs, appointments, caregiver_notes	Schedules and completion logs serve different purposes.

Group	Tables / fields	What to remember
Safety/AI	health_alerts, caregiver_notifications, emergency_events, ai_chatbot_sessions/messages	In-app events, device notices and assistant conversations differ.

What 'AI' means in this project

• **Health Insights:** deterministic comparisons with recent personal averages. **Predictions:** simple linear regression from at least four distinct values; its Low/Moderate/High label describes sample/fit, not clinical certainty. **Anomalies:** fixed review thresholds and personal-baseline deviations.

• **Elle:** Gemini text generation through a server Edge Function. It receives authorized saved context and instructions against diagnosis, prescribing, and unsupported claims. The Gemini key stays on the server. The system has no trained clinical prediction model.

Care and emergency behavior

• Medication/appointment reminders use **expo-notifications** local device scheduling. Daily, twice daily (second time +12h), weekly, and as-needed behavior differ. Editing/deleting cancels old schedules.

• A newly processed out-of-range heart rate (<50 or >110 bpm) or SpO2 (<95%) may show an immediate local warning if permission exists; identical warnings have a one-hour cooldown. This is not guaranteed closed-app server push.

• SOS is manually confirmed, inserts an emergency event, health alert and in-app notification, then opens the saved contact via the phone dialer. No automatic ambulance dispatch is implemented.

• Location is optional foreground **caregiver phone** location saved around authorized sync. The map is the last saved position, not live wearable GPS.

Sources: [S5], [S7], [S8], [S9], [S10], [S19], [S20].

6 What you can demonstrate and what you cannot claim

Claim	Defensible wording
Real time	'Webhook-triggered server synchronization plus Realtime updates while the app is open; timing depends on Google Health delivery and the watch's phone upload.'
Prediction	'A transparent statistical trend from recent distinct samples, not a validated clinical forecast.'
Medical safety	'The app flags configured review thresholds; it does not diagnose, prescribe, or replace emergency services.'
Connected watch	'Google Health reports authorization/paired-device status; this is not a continuous live watch ping.'
Privacy	'Database RLS and server-only secrets protect app records; the current photo bucket is public, so photo URL privacy has a known limitation.'
Deployment	'The repository contains implementation and deployment instructions; the currently deployed revision and active subscriber need a separate live check.'

Suggested 6-minute demo

• 1. Google sign-in and completed onboarding; point out that Health connection is optional.

- 2. Home: identify the patient, read one vital **value and measured time**, then distinguish Last checked/Watch sync time.
- 3. Health: show an insight, prediction pending or result, and anomaly explanation. State sample limitations.
- 4. Care: add a demonstration appointment or medication; show the local reminder permission/result.
- 5. Alerts: show records and the SOS confirmation, but cancel rather than placing a real call.
- 6. Elle: ask a safe caregiver question; show its saved-data context and disclaimer. Then show the admin portal using a separate account and an audit entry if a test environment is available.

Proof to bring to the defense

A build/version label, a redacted test account, a new Google Health sample with its measurement time, a corresponding **vital_sign_logs** row, a Realtime UI update or screen recording, successful RLS denial for an unauthorized caregiver, active/suspended admin examples, function logs for a webhook event, and test output. If these are unavailable, describe the implementation as code evidence only.

Sources: [S3], [S4], [S5], [S14], [S16], [S21].

Terms to define in one sentence

Term	Meaning here
OAuth / PKCE	Authorization with a code exchange bound to the app's original verifier.
JWT	Signed Supabase session token identifying an authenticated user.
Webhook	Google Health's HTTPS change notification to the server.
Realtime	Open app subscription to changes saved in PostgreSQL.
RBAC / RLS	Broad user role and per-row database access rules.
Edge Function	Server-side TypeScript endpoint for secrets and privileged/API work.

Last-minute defense check
Confirm the exact app build, database migration level, deployed Edge Function versions, Google subscriber registration, granted scopes, test account, and device permission state. Redact patient data and secrets from screenshots. If a live service is unavailable, demonstrate the saved record and state the dependency clearly.

7 Likely panel questions - concise answers

Use the first sentence as your spoken answer. Add the second when the panel asks for detail. [S#] maps to code in the final appendix.

Purpose, scope and design

01. What problem does ElderCareAI solve?

It puts available wearable readings and daily caregiver tasks in one app. It helps a caregiver notice changes and organize care, but it does not replace a clinician or emergency service. [S1, S2]

02. Who are the users?

The mobile app serves caregivers linked to an older-adult profile. A separate web portal serves authorized administrators; the older adult is represented in the data model rather than having a separate app role. [S2, S14, S16]

03. What are the main features?

Google sign-in and onboarding; six available health metrics; insights, projections and anomaly cards; medications, appointments, notes, alerts, SOS, Elle, records, location, reports, and an admin portal. [S2, S6-S10]

04. Why Expo, React Native, and Supabase?

The code uses Expo/React Native for the mobile UI and device APIs; Supabase supplies managed Auth, Postgres, RLS, Edge Functions and Realtime. These are implementation choices, not proof of clinical reliability. [S1, S4]

05. Does this pair directly with Fitbit Inspire 3?

No. It reads Google Health API v4 after the watch's phone ecosystem uploads data. ElderCareAI does not directly control Bluetooth pairing or measurement frequency. [S3, S5]

06. Can more than one older adult be linked?

The schema permits linked profiles, but the current sync code selects the caregiver's first older-adult row. I would not claim independent multi-patient wearable routing without further work. [S5]

07. Does the app work offline?

Saved local notification schedules can still fire on the device, but sign-in, fresh health sync, Gemini, Realtime, and Supabase records need network access. The code does not implement a full offline data-sync queue. [S4, S8, S9]

Authentication and Google Health

08. What is the difference between login and Google Health consent?

Login authenticates the caregiver to Supabase. Google Health consent separately authorizes server access to read health data with specific scopes; one does not automatically grant the other. [S3, S11]

09. How does the app login callback work?

Supabase redirects to the app's custom URI with a one-time code; the app exchanges it under a PKCE flow and persists a session. [S3, S4]

10. Why use PKCE and state for Health OAuth?

PKCE binds the authorization code to the original verifier; the short-lived one-time state binds the callback to the initiating caregiver and redirect. The callback consumes state before token exchange. [S11]

11. Where are Google tokens stored?

In google_health_tokens, written by a server function. The table has RLS and no anon/authenticated grants, so ordinary mobile clients cannot read the access or refresh token. [S11, S15]

12. What happens when the Google access token expires?

The sync helper exchanges the stored refresh token for a new access token and updates its expiry. If the authorization cannot be refreshed, the caregiver must reconnect. [S5]

13. What if Health is connected but no readings appear?

Authorization only permits access; the watch must create and upload supported measurements. The sync function distinguishes no data from API errors and the app shows disconnected/empty or stale states. [S5, S6]

Webhook, Realtime and freshness

14. What exactly is a webhook?

It is an HTTPS endpoint that Google Health calls when a subscribed data change occurs. The notification tells our server to fetch current data; it is not itself the health reading shown on screen. [S12, S5]

15. How is the webhook registered?

Deployment must create a Google Health AUTOMATIC subscriber for the Supabase endpoint with the configured secret and supported data types. The repository has the endpoint code and setup instructions; registration in a live Google project needs separate verification. [S12, S22]

16. Why is JWT verification disabled for the callback and webhook?

Google is the caller and does not hold a Supabase user JWT. The callback checks one-time OAuth state and PKCE; the webhook checks its shared secret and Google's event signature. User-triggered sync and admin endpoints use JWT verification. [S11, S12, S13]

17. How does the webhook authenticate Google?

It first compares the Authorization secret. For actual events it verifies Google's signature over the raw request body with a rotating public keyset. Missing or invalid signatures are rejected. [S12, S13]

18. What does the webhook do after verification?

It filters watched UPSERT/DELETE events, deduplicates health user IDs, maps each ID to a stored caregiver token, fetches Health data and updates the linked vital record only when data changed. [S12, S5]

19. Can the webhook work when the app is closed?

Yes, server-side sync can update Postgres while the app is closed. The open app receives changes by Realtime; a closed app loads saved rows on its next opening. That is different from guaranteed push notification delivery. [S12, S6]

20. What if Google sends the same event twice?

The handler deduplicates IDs within a payload and the snapshot comparison avoids writing a new vital just because it checked again. Repeated requests can still occur, so evidence should focus on unchanged values and measurement times. [S12, S5]

21. Is it continuous live heart-rate streaming?

No. The watch and Google Health control when a new sample becomes available. Webhooks and foreground checks react to available data; they do not make the device sample continuously. [S5, S6]

22. What happens if a webhook is missed?

On app launch or foreground return the app silently reconciles, and a caregiver can manually sync. The foreground stale-reading retry policy also checks again, but no fixed update interval is guaranteed. [S6]

23. Why separate measured_at from synced_at?

A server check can be recent while the health measurement is old. Per-metric measurement_times and synced_at let the UI show both truthfully and avoid a false impression of freshness. [S5, S6]

24. What if Google or the server is unavailable?

The sync reports errors or temporary failure and keeps the last saved reading; it does not synthesize a newer value. The webhook returns a failure response when verification or sync cannot complete. [S5, S12]

RBAC, RLS and privacy

25. What is RBAC in this system?

user_access assigns caregiver/admin role and active/suspended status. The app and admin function consult that current database state, so a role is more than a hidden menu choice. [S14, S16, S17]

26. What is RLS and why do you need it?

Postgres Row Level Security checks each database row against the authenticated user's ID and linked patient ownership. It is the database boundary if someone bypasses the app UI and calls the API directly. [S14, S15]

27. Can caregiver A see caregiver B's vitals?

The vital SELECT policy requires a linked older-adult row owned by the current auth.uid(). Active-caregiver status is also required, so another caregiver's patient ID alone is insufficient. [S14, S15]

28. How does suspension work with an old JWT?

The restrictive RLS policy rechecks live user_access status on database access, and the admin function checks current status per request. The app signs out when its own access check sees suspension. [S14, S16, S17]

29. Why a separate admin account?

The portal expects an active admin Auth identity distinct from a caregiver identity. Admins do privileged work only through a server function, keeping the service-role key off the page. [S16, S17]

30. Is hiding /edcadmin/ enough for security?

No. The URL can be guessed. Supabase authentication, current role/status checks, input validation, server-only service credentials and audit records form the access boundary. [S16, S17]

31. What can an admin do?

View dashboards/accounts/monitoring/alerts, update or suspend accounts, manage admins, export bounded CSV reports, and request deletions. The API checks role/status before every action and audits changes/exports. [S16, S18]

32. Can an admin delete or suspend the last admin?

No. The API checks this and a database trigger serializes concurrent changes so the last active admin remains. [S14, S16]

33. What are the report limits?

Admin reports accept at most a 90-day range and return at most 2,000 rows, marking truncation. This bounds export size; it is not a complete archive of all records. [S16, S18]

34. Are all records private?

Caregiver tables and Google tokens are access-controlled, but the existing profile-photo bucket is public. A known photo URL can remain reachable; that limitation should be disclosed. [S15, S17]

35. Where do you keep API keys?

The mobile and static page contain only a public Supabase URL/publishable key. Google OAuth client secret, Gemini key and service-role key belong in server-side Edge Function secrets. [S4, S11, S16, S19]

AI, alerts, location and records

36. Is this a trained medical AI model?

No. Insights, predictions and anomalies are deterministic calculations. Elle is a separate Gemini-based text assistant. Neither is medically validated for diagnosis or treatment. [S7, S19]

37. How are predictions calculated?

For heart rate, sleep or steps, the code takes at least four distinct recent values, fits a simple linear regression, and labels confidence from sample count and fit. It is a short-term statistical projection. [S7]

38. How are anomalies detected?

The app checks configured thresholds - SpO2 below 95%, heart rate below 50 or above 110, sleep below 5 hours, HRV below 30 ms - plus two-standard-deviation changes for supported personal baselines. [S7]

39. Does Elle get arbitrary patient data?

Its Edge Function verifies the caregiver and assembles bounded saved context such as profile, vitals, medications, appointments, notes and alerts. Its prompt forbids diagnosis/prescribing, though generated answers still require human review. [S19]

40. What if Gemini returns an unsafe or wrong answer?

The prompt sets a non-diagnostic boundary and the app should present it as decision support. A language model can still err; critical care decisions require checking source records and a qualified professional. [S19]

41. Are alerts the same as phone notifications?

No. health_alerts and caregiver_notifications are saved database records. Medication and appointment reminders are local device schedules; unusual-reading warnings are local when the app processes a new row and has permission. [S8, S9]

42. Does SOS automatically contact emergency services?

No. A caregiver confirms SOS, which saves records and opens a call to the stored emergency contact. The user must complete the call; local emergency services remain separate. [S8]

43. Does the app track the older adult live?

No. The optional location is the caregiver phone's foreground location at a sync, shown as last known sync location. It is not continuous GPS or the watch's current position. [S6, S10]

44. What is the difference between a medical record and a health metric?

Health metrics come from synchronized Google Health samples. Medical history and checkups are caregiver-entered app records; they need confirmation against clinical documentation. [S10]

45. How does report export work?

The mobile app formats saved vitals or medication schedules as a PDF using expo-print and opens the device sharing flow. Admin exports are separate bounded CSV-style data reports through the portal API. [S10, S16]

Testing, deployment and limitations

46. How did you test the system?

The repository has Jest/React Native Testing Library suites, a TypeScript typecheck command and documented integration cases. Mocked tests validate logic and UI interactions; live OAuth, watch, webhook and permissions need device/deployment evidence. [S1, S21]

47. How would you prove the webhook end to end?

Capture a new watch reading and Google Health timestamp, a verified webhook/function log, a newer measurement_times value in vital_sign_logs, and the open app update without tapping refresh. Also test unauthorized and invalid-signature requests. [S5, S6, S12]

48. What if the watch is disconnected?

The app retains the latest saved value with its old measured time and can show delayed sync guidance. It cannot force the external watch/phone to reconnect or fabricate data. [S5, S6]

49. What if notification permission is denied?

The scheduling function returns without a device reminder and the user can enable permission in Settings. Saved care items still exist in Supabase. [S9]

50. What are the main limitations?

External upload latency, first-profile Health routing, no full offline queue, local rather than guaranteed server push alerts, public profile-photo URLs, and non-clinical analysis. Deployment also depends on active Google Health subscriber/scopes and server secrets. [S5-S7, S9, S15, S22]

51. What would you improve next?

Add explicit per-device/per-patient Health mapping, private signed photo access, monitored webhook delivery and retry/idempotency telemetry, stronger clinical evaluation of thresholds, and an offline data strategy. These are proposed improvements, not current features. [S5, S12, S15]

8 Source map and verification checklist

The references below identify the repository files used to check this guide. Paths are relative to the repository root. The older database design drawing is useful for presentation, but current SQL migrations govern access and later schema changes.

Ref	Code / documentation anchor
S1	package.json; app.json; eas.json; README.md
S2	app/(tabs)/; app/setup/; app/health-records.tsx; docs/ELDERCAREAI_USER_GUIDE.md
S3	src/providers/AuthProvider.tsx; src/lib/auth-callback.ts; src/lib/onboarding-route.ts
S4	src/lib/supabase.ts; app/_layout.tsx
S5	supabase/functions/_shared/google-health-sync.ts; health-snapshot.ts
S6	src/providers/HealthDataProvider.tsx; src/lib/automatic-health-sync.ts; watch-sync.ts
S7	src/lib/health-analysis.ts; src/lib/sleep-score.ts
S8	app/(tabs)/alerts.tsx; app/(tabs)/care.tsx
S9	src/lib/notifications.ts
S10	app/location.tsx; app/health-records.tsx; src/lib/pdf-reports.ts
S11	supabase/functions/google-health-oauth-start/index.ts; google-health-oauth-callback/index.ts
S12	supabase/functions/google-health-webhook/index.ts; _shared/health-webhook.ts
S13	supabase/functions/_shared/health-signature.ts; supabase/config.toml
S14	supabase/migrations/20261011000000_admin_access.sql
S15	supabase/migrations/20260922010000_add_onboarding.sql; 20260922020000_app_rls.sql
S16	supabase/functions/admin-portal/index.ts; _shared/admin-security.ts
S17	website/edcadmin/admin.js; website/edcadmin/README.md
S18	website/edcadmin/index.html
S19	supabase/functions/ai-care-assistant/index.ts
S20	supabase/migrations/20261008000000_vital_measurement_times.sql; 20261010010000_move_sleep_history_to_vital_logs.sql
S21	__tests__/; integration testing/; docs/UNIT_TEST_EVIDENCE.md; docs/INTEGRATION_TEST_EVIDENCE.md
S22	README.md, Google Health subscriber deployment instructions